

- DevSecOps Architect — Interview Questions & Answers
 - Section 1: CI/CD Pipeline Security (Q1–Q6)
 - Q1. Walk us through the security scanning stages in your CI pipeline and explain the order.
 - Q2. How does GitHub OIDC work for keyless AWS authentication, and why is it better than storing access keys?
 - Q3. Explain the GitOps deployment model with ArgoCD. What happens when a developer pushes code?
 - Q4. Why use commit SHA as image tags instead of latest or semantic versioning?
 - Q5. How do you handle CI/CD pipeline failures? What if the SonarCloud quality gate fails?
 - Q6. What are DORA metrics and how do you measure them in your pipeline?
 - Section 2: Kubernetes Security & Architecture (Q7–Q13)
 - Q7. Explain your Pod Security Standards implementation.
 - Q8. Explain the NetworkPolicy strategy and why "default deny" matters.
 - Q9. How do ResourceQuotas and LimitRanges protect the cluster?
 - Q10. Explain topologySpreadConstraints and why they matter for HA.
 - Q11. How does Karpenter differ from Cluster Autoscaler, and why did you choose it?
 - Q12. How do liveness and readiness probes work, and what's the difference?
 - Q13. Explain IRSA (IAM Roles for Service Accounts) and why it matters.
 - Section 3: Container Security (Q14–Q17)
 - Q14. Walk through the Dockerfile security best practices you implemented.
 - Q15. Why use -XX:MaxRAMPercentage=75.0 instead of -Xmx?
 - Q16. What is the risk of using imagePullPolicy: Always vs IfNotPresent?
 - Q17. How would you implement container image signing and verification?
 - Section 4: Infrastructure as Code & Network Security (Q18–Q22)
 - Q18. Explain the VPC architecture and why you use 6 VPC Endpoints.
 - Q19. Why is the EKS API server private, and how do you access it?
 - Q20. Explain the defense-in-depth security layers from client to pod.
 - Q21. How does WAF protect against the OWASP Top 10?
 - Q22. How do you handle multi-region DR in the EKS project?
 - Section 5: Compliance & Governance (Q23–Q26)

- Q23. How does your compliance-check workflow map to PCI-DSS, HIPAA, and SOC 2?
- Q24. How do you ensure secrets are never exposed in your pipeline?
- Q25. What is IMDSv2 and why is it critical for EKS security?
- Q26. How would you implement runtime security scanning in production?
- Section 6: Observability & Operations (Q27–Q30)
 - Q27. How do you monitor a Java application running on EKS?
 - Q28. How does HPA work with Karpenter for autoscaling?
 - Q29. A pod is in CrashLoopBackOff. Walk through your debugging process.
 - Q30. How would you implement a zero-downtime deployment for a Java app on EKS?

DevSecOps Architect — Interview Questions & Answers

Based on the **Java_DevSecOps** and **EKS_DevSecOps** projects. Covers CI/CD, Kubernetes security, IaC, container hardening, compliance, and observability.

Section 1: CI/CD Pipeline Security (Q1–Q6)

Q1. Walk us through the security scanning stages in your CI pipeline and explain the order.

Answer: Our CI pipeline has **6 security stages** in deliberate order:

1. **Gitleaks (Secrets Scan)** — Runs first, before any build. Scans entire git history for leaked AWS keys, tokens, passwords. If secrets are found, the pipeline fails immediately — no point building compromised code.
2. **Compile + Unit Tests with JaCoCo** — Establishes code correctness and coverage baseline.

3. **SonarCloud (SAST)** — Static Application Security Testing. Finds injection flaws, XSS, null pointers, code smells. Uses JaCoCo coverage data from the previous step.
4. **OWASP Dependency-Check (SCA)** — Software Composition Analysis. Scans all Maven dependencies (including transitive) against the NVD for known CVEs.
5. **Docker Build** — Only after code passes SAST/SCA.
6. **Trivy (Container Image Scan)** — Scans the built Docker image for OS package and application vulnerabilities. Results uploaded as SARIF to GitHub Security tab.

Why this order: Fail fast on cheap checks (secrets scan = seconds) before expensive ones (SAST = minutes). Each stage gates the next — no point scanning an image if the code has critical CVEs.

Q2. How does GitHub OIDC work for keyless AWS authentication, and why is it better than storing access keys?

Answer: In `github-oidc.tf`, we configure an **OpenID Connect identity provider** in AWS IAM:

1. GitHub Actions generates a short-lived JWT during each workflow run
2. The workflow calls `sts:AssumeRoleWithWebIdentity` with this JWT
3. AWS validates the JWT against GitHub's OIDC provider (`token.actions.githubusercontent.com`)
4. AWS issues **temporary credentials** (15-min default) scoped to the IAM role

Trust policy is scoped tightly:

```
"token.actions.githubusercontent.com:sub" =  
"repo:ORG/REPO:ref:refs/heads/main"
```

Only the specific repo + main branch can assume the role. PRs cannot push images.

Why better than stored keys:

- No long-lived secrets to rotate, leak, or steal
- Credentials are temporary and auto-expire

- IAM role is scoped to minimum permissions (ECR push only)
 - Eliminates the #1 cause of AWS breaches: compromised access keys
-

Q3. Explain the GitOps deployment model with ArgoCD. What happens when a developer pushes code?

Answer: The flow is:

1. **Developer pushes to `main`** → CI pipeline triggers
2. **CI builds, scans, pushes image to ECR** with commit SHA tag (immutable)
3. **CI triggers CD workflow** via `repository_dispatch`
4. **CD workflow updates `deployment.yaml`** in the manifest repo with the new image tag using `sed`
5. **ArgoCD detects the Git change** (polls every ~3 min) and syncs the cluster

Key ArgoCD settings from `application.yaml`:

- `selfHeal: true` — Reverts any manual cluster changes (drift detection)
- `prune: true` — Deletes orphaned resources removed from Git
- `ApplyOutOfSyncOnly: true` — Only touches changed resources
- Retry with exponential backoff (5 attempts, 5s→3m)

Critical principle: The CD pipeline **never runs `kubectl`**. It only modifies Git state.

ArgoCD is the single actor that touches the cluster — ensuring full audit trail and single source of truth.

Q4. Why use commit SHA as image tags instead of `latest` or semantic versioning?

Answer:

- **Immutability** — A commit SHA uniquely identifies the exact code that built the image. You can never accidentally overwrite it.
- **Traceability** — From a running pod, you can trace back to the exact commit, PR, and CI run.

- **Rollback precision** — `kubectl set image ... :abc123` vs `kubectl set image ... :v2.3.1` — the SHA tells you exactly what's running.
- **ArgoCD compatibility** — ArgoCD detects the tag change in Git and syncs. With `latest`, the tag doesn't change even when the image does.

We also push `latest` as a convenience tag for local development, but production always uses the SHA.

Q5. How do you handle CI/CD pipeline failures? What if the SonarCloud quality gate fails?

Answer:

- **SonarCloud quality gate:** Set `sonar.qualitygate.wait=true` — the pipeline blocks until SonarCloud returns pass/fail. A failure stops the pipeline before Docker build, saving compute time.
- **OWASP DC:** Currently runs with `|| true` (soft fail) because NVD can have false positives. In production, I'd set a CVSS threshold (e.g., fail on CRITICAL only).
- **Trivy:** `exit-code: '0'` (soft fail). Results go to GitHub Security tab for triage. In production, set to `'1'` for CRITICAL/HIGH.
- **Gitleaks:** Hard fail. No exceptions — leaked secrets must be remediated immediately.

The philosophy: Hard-fail on things that are always wrong (leaked secrets). Soft-fail on things that need human judgment (CVE triage). Always capture results as artifacts for review.

Q6. What are DORA metrics and how do you measure them in your pipeline?

Answer: DORA (DevOps Research and Assessment) defines four key metrics:

Metric	How We Measure	Elite Target
Deployment Frequency	Count GitHub Deployments API entries per week	≥1/day
Lead Time for Changes	Commit timestamp → deployment timestamp (recorded in CI job)	<1 hour
Change Failure Rate	Failed deployment statuses / total deployments	<15%
Mean Time to Recovery	Time to close issues labeled incident	<1 hour

Implementation: The **dora-report.yml** workflow runs weekly, queries the GitHub API, calculates all 4 metrics, and creates a GitHub Issue with the report. The CI pipeline records lead time per deployment. **Zero extra infrastructure** — pure GitHub API.

Section 2: Kubernetes Security & Architecture (Q7–Q13)

Q7. Explain your Pod Security Standards implementation.

Answer: In the EKS project's **security.yaml**, the namespace uses **Pod Security Admission** (PSA) at the **restricted** level:

```
labels:
  pod-security.kubernetes.io/enforce: restricted
  pod-security.kubernetes.io/audit: restricted
  pod-security.kubernetes.io/warn: restricted
```

This enforces:

- **runAsNonRoot: true** — No root containers
- **allowPrivilegeEscalation: false** — No privilege escalation

- `readOnlyRootFilesystem` — Immutable filesystem
- No `hostNetwork`, `hostPID`, `hostIPC`
- Drop all capabilities except `NET_BIND_SERVICE`

Three modes: `enforce` (blocks), `audit` (logs), `warn` (warns user). We use all three at `restricted` for maximum protection.

Q8. Explain the NetworkPolicy strategy and why "default deny" matters.

Answer: From `security.yaml`:

1. **Default deny all** — `podSelector: {}` with both Ingress and Egress policy types.
This means **no pod can talk to anything** unless explicitly allowed.
2. **Allow ALB → App on 8080** — Only TCP/8080 ingress from ALB
3. **Allow App → Aurora on 3306** — Only MySQL egress + DNS (53/TCP+UDP) + HTTPS (443) for AWS API calls

Why default deny: Without it, every pod can talk to every other pod. A compromised pod could lateral-move to the database, monitoring stack, or other services. Default deny implements **zero-trust networking** — every communication path must be explicitly declared.

DNS exception is critical: Without ports 53/TCP and 53/UDP in egress, pods can't resolve service names. This is the most common mistake when implementing deny-all policies.

Q9. How do ResourceQuotas and LimitRanges protect the cluster?

Answer: From `security.yaml`:

ResourceQuota — namespace-level caps:

- Max 20 CPU cores requested, 40 limits
- Max 40Gi memory requested, 80Gi limits

- Max 50 pods, 10 services

LimitRange — per-container defaults:

- Default request: 100m CPU, 128Mi memory
- Default limit: 500m CPU, 512Mi memory

Why both:

- **ResourceQuota prevents noisy neighbors** — A runaway deployment can't consume the entire cluster
- **LimitRange catches missing specs** — If a developer forgets to set resource limits, the defaults kick in instead of unlimited
- **Together they enforce QoS** — Pods get **Burstable** QoS class, which means they're guaranteed their requests but can burst up to limits

Q10. Explain topologySpreadConstraints and why they matter for HA.

Answer: From **deployment.yaml**:

```
topologySpreadConstraints:  
  - maxSkew: 1  
    topologyKey: topology.kubernetes.io/zone  
    whenUnsatisfiable: DoNotSchedule
```

This distributes pods **evenly across availability zones**. With 2 replicas across 3 AZs, you get 1 pod per AZ (maxSkew=1 means no AZ can have more than 1 extra pod compared to others).

DoNotSchedule means: **refuse to schedule** if it would violate the constraint. This is stricter than **ScheduleAnyway**.

Why it matters: Without this, the scheduler might place both replicas in the same AZ. If that AZ has an outage, you lose 100% capacity. With topology spread, you're guaranteed to survive a single AZ failure.

Q11. How does Karpenter differ from Cluster Autoscaler, and why did you choose it?

Answer: **Cluster Autoscaler** scales node groups. **Karpenter** provisions individual nodes.

Feature	Cluster Autoscaler	Karpenter
Granularity	Node group level	Individual nodes
Instance selection	Fixed instance type per group	Auto-selects optimal instance from pool
Scale-up speed	2–5 minutes	30–60 seconds
Spot support	Manual ASG configuration	Built-in with consolidation
Right-sizing	Limited	Bins pods onto smallest possible instance

Why Karpenter: It provisions **right-sized nodes** based on pending pod requirements. If a pod needs 4Gi RAM, Karpenter picks the cheapest instance that fits — not a fixed **t3.medium**. Combined with Spot instances, this significantly reduces compute costs.

Q12. How do liveness and readiness probes work, and what's the difference?

Answer: From **deployment.yaml**:

Readiness probe — "Is this pod ready to receive traffic?"

- **initialDelaySeconds: 30** — Wait 30s for Spring Boot startup
- **periodSeconds: 10** — Check every 10s
- If failing: Pod is **removed from the Service** (no traffic routed to it)

Liveness probe — "Is this pod healthy?"

- **initialDelaySeconds: 60** — Longer delay (Spring Boot can take time)
- **periodSeconds: 15** — Check every 15s
- If failing 3 times: Pod is **killed and restarted**

Both hit `/actuator/health` on port 8080.

Key distinction: A pod can be live but not ready (e.g., warming up cache). Removing readiness before liveness ensures zero-downtime deployments — the old pod keeps serving until the new pod is ready.

Q13. Explain IRSA (IAM Roles for Service Accounts) and why it matters.

Answer: IRSA maps a **Kubernetes ServiceAccount** to an **AWS IAM Role**. Instead of giving the entire node IAM permissions, each pod gets only the permissions it needs.

How it works:

1. Create an IAM role with a trust policy for the EKS OIDC provider
2. Annotate the K8s ServiceAccount with `eks.amazonaws.com/role-arn`
3. The EKS Pod Identity webhook injects AWS credentials into the pod

Example from our project: The ALB Ingress Controller's ServiceAccount is annotated with an IAM role that only has `elasticloadbalancing:*` permissions. The application pods have no AWS permissions at all.

Why it matters: Without IRSA, all pods on a node share the node's IAM role. A compromised app pod could access ECR, S3, or other AWS services it shouldn't. IRSA enforces **least privilege at the pod level**.

Section 3: Container Security (Q14–Q17)

Q14. Walk through the Dockerfile security best practices you implemented.

Answer: From `app/Dockerfile`:

1. **Multi-stage build** — Builder stage has Maven+JDK (800MB). Runtime stage has only JRE (200MB). Source code, build tools, Maven cache never reach the final

image.

2. **Non-root user** — `groupadd appgroup && useradd appuser`, then `USER appuser`. Prevents container escape from gaining root.
 3. **Minimal base image** — `eclipse-temurin:17-jre-jammy` — JRE only, no compiler, no dev tools.
 4. **Layer caching** — `COPY pom.xml + mvn dependency:go-offline` before `COPY src`. Dependencies are cached unless `pom.xml` changes.
 5. **HEALTHCHECK** — Docker-level health monitoring independent of K8s probes.
 6. **JVM container awareness** — `-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0` — JVM respects cgroup memory limits.
 7. **Non-blocking entropy** — `-Djava.security.egd=file:/dev/./urandom` — Faster startup in containers (no blocking on `/dev/random`).
-

Q15. Why use `-XX:MaxRAMPercentage=75.0` instead of `-Xmx`?

Answer: `-Xmx512m` is a fixed value. If the container limit changes (256Mi → 1Gi), you must update both the K8s manifest AND the Dockerfile.

`-XX:MaxRAMPercentage=75.0` is **dynamic** — JVM calculates max heap as 75% of the container's memory limit. Change the K8s resource limit, and the JVM automatically adjusts.

Why 75% and not 100%? The JVM needs memory beyond heap: metaspace, thread stacks, native memory, GC overhead. 75% leaves ~25% for non-heap memory. Going higher risks OOMKilled pods.

Q16. What is the risk of using `imagePullPolicy: Always` vs `IfNotPresent`?

Answer: We use `Always` because:

- With mutable tags (like `latest`), `IfNotPresent` might use a stale cached image
- Ensures the pod always runs the exact image from the registry

- Combined with SHA tags, this guarantees consistency

Trade-off: **Always** means every pod restart hits ECR. In a private EKS cluster, this goes through VPC Endpoints (no internet), so latency is minimal. But if ECR is down, pods can't restart — mitigated by the ECR lifecycle policy keeping recent images.

Q17. How would you implement container image signing and verification?

Answer: I'd use **AWS Signer** + **Kyverno/OPA Gatekeeper**:

1. **Sign:** After Trivy scan passes in CI, sign the image with AWS Signer (or Cosign/Sigstore)
2. **Verify:** Deploy a Kyverno **ClusterPolicy** that requires valid signatures before allowing pod creation
3. **Enforce:** Any unsigned or tampered image is rejected by the admission controller

This closes the gap between "image was scanned" and "image running in cluster is the same one that was scanned."

Section 4: Infrastructure as Code & Network Security (Q18–Q22)

Q18. Explain the VPC architecture and why you use 6 VPC Endpoints.

Answer: From **vpc.tf**:

- **3 public subnets** (ALB, NAT Gateway, Bastion) — one per AZ
- **3 private subnets** (EKS nodes, pods) — one per AZ

6 VPC Endpoints: ECR API, ECR DKR, S3 (gateway), STS, CloudWatch Logs, EKS API.

Why: Private EKS nodes have **no internet access** (no IGW route). Without VPC Endpoints, nodes can't pull images from ECR, send logs to CloudWatch, or authenticate

to AWS. VPC Endpoints create **PrivateLink connections** that keep all traffic within the AWS network.

S3 endpoint is a Gateway type (free). The others are Interface type (~7/month each). This is a cost trade-off—36/month for endpoints vs. the security risk of routing traffic through the internet.

Q19. Why is the EKS API server private, and how do you access it?

Answer: Private API server means **kubectl** commands only work from within the VPC. No internet-facing Kubernetes API.

Access path:

```
Developer → SSH to Bastion (public subnet) → kubectl → EKS API (private endpoint)
```

Why private:

- Eliminates the entire class of K8s API attacks from the internet
- No exposed attack surface for unauthenticated enumeration
- Combined with AWS IAM authentication (not just kubeconfig), you need both SSH access to bastion AND valid AWS IAM credentials

Trade-off: CI/CD can't directly **kubectl apply**. That's why we use ArgoCD (runs inside the cluster) instead of direct deployment from GitHub Actions.

Q20. Explain the defense-in-depth security layers from client to pod.

Answer: 12 layers from the Java_DevSecOps project:

#	Layer	Protection
1	Route53 + ACM	TLS 1.3, certificate validation

#	Layer	Protection
2	WAF v2	OWASP Top 10, SQLi, rate limiting (2000 req/5min), known bad inputs
3	API Gateway	JWT authorization, request throttling
4	Cognito	User pools, MFA, token revocation
5	ALB + VPC Link	Private connectivity, health checks
6	Private subnets	No public IPs on EKS nodes
7	Private EKS API	kubectl only from VPC
8	IMDSv2	Blocks SSRF credential theft
9	KMS	Kubernetes secrets encrypted at rest
10	IRSA	Fine-grained IAM per service account
11	GitHub OIDC	Keyless CI/CD auth
12	SonarCloud + Trivy	SAST, SCA, container scanning

Key insight: Each layer is independent. Compromising WAF doesn't bypass Cognito. Compromising a pod doesn't give AWS IAM access (IRSA). This is the **Swiss cheese model** — holes in individual layers don't align.

Q21. How does WAF protect against the OWASP Top 10?

Answer: From `waf.tf`, four rule groups in priority order:

1. **Rate limiting** (priority 1) — Blocks IPs exceeding 2000 requests/5 min. Stops DDoS and brute force.
2. **AWSManagedRulesCommonRuleSet** (priority 2) — Covers XSS, LFI, RFI, path traversal, and other OWASP Top 10 patterns.
3. **AWSManagedRulesSQLiRuleSet** (priority 3) — SQL injection detection in URLs, query params, headers, and body.
4. **AWSManagedRulesKnownBadInputsRuleSet** (priority 4) — Blocks Log4j exploits, SSRF patterns, and known malicious payloads.

All rules have **CloudWatch metrics enabled** for monitoring and tuning. Default action is **allow** — only matched malicious patterns are blocked.

Q22. How do you handle multi-region DR in the EKS project?

Answer: From the EKS_DevSecOPs architecture:

- **Primary:** us-east-1 with EKS + Aurora Writer
- **Secondary:** ap-south-1 with EKS + Aurora Reader
- **Route53 failover routing** — Health checks on primary ALB; automatic DNS failover to secondary
- **Aurora Global Database** — Cross-region replication with <1s RPO
- **Identical EKS clusters** — Same Helm charts, same Karpenter config

Failover process:

1. Route53 detects primary ALB health check failure
2. DNS automatically resolves to secondary region ALB
3. Aurora promotes secondary reader to writer
4. Secondary EKS cluster serves traffic

Terraform approach: Separate environments (**primary/** and **secondary/**) sharing the same modules. The Terraform CI pipeline plans and applies both regions sequentially — primary first, then secondary.

Section 5: Compliance & Governance (Q23–Q26)

Q23. How does your compliance-check workflow map to PCI-DSS, HIPAA, and SOC 2?

Answer: Four scanning stages:

1. **Checkov** — Scans Terraform against specific CKV checks mapped to PCI-DSS (encryption, access controls), HIPAA (audit logging), SOC 2 (change management)
2. **tfsec** — Additional Terraform security checks for AWS best practices
3. **Kubescape** — Scans K8s manifests against NSA/CISA, MITRE ATT&CK, and SOC 2 frameworks
4. **Hadolint** — Dockerfile linting against CIS Docker Benchmark

Custom validation checks:

- PCI-DSS 6.5.1: `grep "privileged: true"` — No privileged containers
- PCI-DSS 7.1: `grep "runAsNonRoot: true"` — Least privilege
- PCI-DSS 6.3.2: `grep "password|secret"` in Dockerfile — No hardcoded secrets
- HIPAA 164.312(e): Read-only filesystem check
- SOC 2 CC8.1: Multi-stage build check

Output: Auto-generated compliance report created as a GitHub Issue with pass/fail status per control.

Q24. How do you ensure secrets are never exposed in your pipeline?

Answer: Multiple layers:

1. **Gitleaks** — First CI step. Scans entire git history for accidentally committed secrets.
 2. **GitHub OIDC** — No AWS access keys stored in GitHub Secrets at all.
 3. `.env` in `.gitignore` — API keys never committed to the repo.
 4. **PCI-DSS 6.3.2 check** — Compliance workflow greps Dockerfile for hardcoded secrets.
 5. **KMS encryption** — Kubernetes secrets encrypted at rest in etcd.
 6. **IRSA** — Pod-level IAM, no shared node credentials.
 7. **ECR private registry** — Images accessed via VPC Endpoint, never public.
-

Q25. What is IMDSv2 and why is it critical for EKS security?

Answer: IMDS (Instance Metadata Service) is how EC2 instances access their IAM credentials. **IMDSv1** allows simple GET requests — an SSRF vulnerability in your app can steal node credentials.

IMDSv2 requires a PUT request to get a session token first, then uses that token in subsequent requests. This blocks SSRF-based credential theft because:

- SSRF can follow redirects but can't make PUT requests with custom headers
- The token has a short TTL and is bound to the instance

In our EKS config, nodes are launched with `http_tokens = "required"` (IMDSv2 only). Combined with IRSA, even if IMDS is somehow accessed, the node role has minimal permissions.

Q26. How would you implement runtime security scanning in production?

Answer: I'd add three tools:

1. **Falco** — Runtime threat detection. Monitors syscalls for suspicious behavior (shell spawned in container, unexpected network connections, file access in /etc). Deployed as DaemonSet on every node.
 2. **AWS GuardDuty for EKS** — Detects K8s API anomalies, crypto-mining, DNS exfiltration. Already referenced in the EKS project's monitoring stack.
 3. **Admission Controllers** — Kyverno or OPA Gatekeeper to enforce policies at deploy time (no `latest` tags, mandatory labels, image from trusted registry only).
-

Section 6: Observability & Operations (Q27–Q30)

Q27. How do you monitor a Java application running on EKS?

Answer: Three pillars:

Metrics: Prometheus scraping via annotations:

```
prometheus.io/scrape: "true"
prometheus.io/port: "8080"
prometheus.io/path: "/actuator/metrics"
```

Spring Boot Actuator exposes JVM heap, GC, thread pools, HTTP request latency, and custom business metrics.

Logs: Application logs → stdout → Container runtime → CloudWatch Logs (via Fluent Bit DaemonSet or CloudWatch agent).

Traces: Spring Boot + OpenTelemetry or AWS X-Ray SDK for distributed tracing across microservices.

Alerting: CloudWatch Alarms on key metrics (CPU >80%, 5xx error rate >1%, pod restart count). SNS notifications to Slack/PagerDuty.

Q28. How does HPA work with Karpenter for autoscaling?

Answer: Two-level autoscaling:

1. **HPA (Horizontal Pod Autoscaler)** — Watches CPU/memory metrics. When average CPU >70%, it adds more pod replicas.
2. **Karpenter** — Watches for unschedulable pods (pending). When HPA creates pods that can't be scheduled (no node capacity), Karpenter provisions a right-sized node in ~60 seconds.

Scale-up flow:

```
Load increases → HPA adds pods → Pods go Pending → Karpenter provisions node
→ Pods scheduled
```

Scale-down flow:

Load decreases → HPA removes pods → Node underutilized → Karpenter consolidates (moves pods, terminates node)

Q29. A pod is in CrashLoopBackOff. Walk through your debugging process.

Answer:

1. `kubectl describe pod <name>` — Check Events, exit codes, last state
 2. `kubectl logs <pod> --previous` — Check logs from the crashed container
 3. Common Java causes:
 - **Exit code 137:** OOMKilled. Check `resources.limits.memory` — too low for JVM + non-heap. Increase or tune `MaxRAMPercentage`.
 - **Exit code 1:** Application error. Check Spring Boot startup logs for missing env vars, DB connection failures, or config errors.
 - **Readiness probe failing:** App starts but `/actuator/health` returns DOWN. Check `initialDelaySeconds` — may need to increase for slow Spring Boot startup.
 4. `kubectl get events --sort-by='.lastTimestamp'` — Cluster-level events (image pull failures, resource quota exceeded)
 5. `kubectl exec -it <pod> -- /bin/sh` — If the container stays up long enough, exec in and inspect
-

Q30. How would you implement a zero-downtime deployment for a Java app on EKS?

Answer: Already implemented via multiple mechanisms:

1. **Rolling update strategy:** `maxSurge: 1, maxUnavailable: 0` — Always at least N replicas running during deployment.
2. **Readiness probes:** New pod only receives traffic after `/actuator/health` returns 200. Old pod keeps serving until new pod is ready.
3. **Topology spread:** Pods spread across AZs — deployment rolls one AZ at a time.

4. **Graceful shutdown:** `terminationGracePeriodSeconds: 30` — Pod gets 30s to finish in-flight requests before SIGKILL.
5. **PodDisruptionBudget (PDB):** Would add `minAvailable: 1` to prevent voluntary disruptions from terminating all pods.
6. **ArgoCD progressive sync:** `ApplyOutOfSyncOnly: true` — Only changes affected resources.

Spring Boot side: Register a shutdown hook to stop accepting new requests and drain the connection pool before exit.

Tip for the interview: For each answer, be ready to point to the exact file in your project. Interviewers value candidates who can say "line 59 of `deployment.yaml`" not just "we use topology spread."